Software Requirements Specification (SRS) for University of Hyderabad Placement Management System

1. Introduction

1.1 Purpose

This SRS defines the requirements for a centralized web-based placement portal for the University of Hyderabad. Its purpose is to document all functional and non-functional requirements of the Placement Management System, providing a single source of truth for designers, developers, testers, and stakeholders. The SRS follows industry and IEEE standards, ensuring clarity and completeness. It covers features such as user authentication, job postings, application processing, scheduling, notifications, reporting, and analytics, and specifies system behaviors under various scenarios. The goal is to eliminate ambiguity: every requirement is stated precisely, without design details. The system will serve students, faculty, administrators (TPOs/HoDs), and company recruiters, automating workflows that are currently manual or fragmented.

1.2 Scope

The scope of the project is the development of a **Placement Management Portal** that replaces disparate tools (spreadsheets, emails, manual tracking) with an integrated web platform. The portal will support:- **Company Management:** Recruiters register and post job openings with eligibility criteria (branches, CGPA, etc.).- **Student Management:** Students register, create profiles and resumes, view eligible jobs, apply online, and track application status.- **Faculty Module:** Faculty can endorse students, post academic announcements, and view placement statistics of their department.- **Admin Module:** Admins (Placement Officers/Department Heads) manage users (add/edit students, faculty, companies), approve job postings, schedule placement drives, and generate analytics.- **Scheduling & Interviews:** Automate scheduling of placement rounds (tests/interviews), matching student availability, and tracking results.- **Notifications:** Automatic emails/SMS alerts for events (drive announcements, interview schedules, results).- **Reporting:** Dashboards for KPI tracking (number of drives, applications, placement rates) for administrators and faculty.

The system will be accessible via modern web browsers, with responsive design for mobile devices. It will use a three-tier architecture (frontend UI, backend API server, and database). Authentication may leverage Firebase Authentication or an equivalent (e.g. JWT/OAuth2). Data storage will use a cloud-based database (e.g. PostgreSQL or Firebase Firestore). This SRS does *not* cover hardware procurement, user training, or future mobile app development, except where noted in “Assumptions”.

Following IEEE/ISO practices, this document treats software requirements as necessary and unambiguous statements. The SRS is organized into functional requirements (what the system must do) and non-functional requirements (how the system should perform). All requirements are independently verifiable. Functional requirements are documented in section 3, and non-functional attributes (security, performance, etc.) are also specified. The SRS will be the basis for design and testing.

1.3 Definitions, Acronyms, and Abbreviations

API: Application Programming Interface.

Auth0: A cloud service for authentication (alternative to Firebase Auth).

AWS/Azure: Cloud hosting providers (Amazon Web Services, Microsoft Azure).

Backend: Server-side application logic (e.g. Node.js/Express).

CGPA: Cumulative Grade Point Average.

CRM: Customer Relationship Management.

DR: Disaster Recovery.

GCP: Google Cloud Platform.

HoD: Head of Department.

JWT (JSON Web Token): A token-based authentication method.

RBAC (Role-Based Access Control): Access control based on user roles.

REST: Representational State Transfer, a type of web API.

RBMS: (Note: if used, e.g. Relational Database Management System).

TPO: Training and Placement Officer.

UI: User Interface.

UX: User Experience.

WCAG: Web Content Accessibility Guidelines.

(All other terms are used as commonly understood in university and software contexts.)

1.4 References

IEEE Std 29148-2018 (ISO/IEC/IEEE 29148:2018) – Systems and software engineering – Requirements engineering.

Karl E. Wiegers, *"Software requirements specification and IEEE standards"*, TechTarget, 2007[3][1].

Wikipedia, *"Software requirements specification"*, Section on quality characteristics[2].

Google Firebase Documentation: *"General best practices for setting up Firebase projects"*[4].

Google Firebase Documentation: *"Cloud Firestore"* (preferred database for scalability)[5].

BMC Software Blog, *"Software Requirements Specification (SRS) – How to Write SRS with Examples"*[6].

2. Overall Description

2.1 Product Perspective

The Placement Portal is a new, standalone web application. It replaces the current manual or disparate placement processes of the University of Hyderabad. It is not an extension of an existing system but may integrate with external services. The system will follow a three-tier architecture: a responsive **web frontend** (built with React.js or similar), an **API/backend** (Node.js/Express or Django/Python), and a **database** (relational like PostgreSQL or NoSQL like Firebase Firestore).

All components will communicate over HTTP/HTTPS using RESTful APIs. Authentication and authorization will be enforced at the API layer, e.g. via Firebase Authentication or JSON Web Tokens (JWT). Where applicable, the portal will integrate with university services (such as LDAP/Active Directory for initial user import, if permitted) and third-party services (email/SMS gateways for notifications). All client-side and server-side modules run on Cloud infrastructure (AWS/Azure/GCP) to ensure high availability. For example, Google Firebase allows registering all apps (frontend, admin, etc.) under one project so they share resources like Authentication and Firestore[4].

Figure 1 (not shown) would depict a high-level architecture: users (students, faculty, admin, companies) access the web UI, which calls backend APIs, which in turn interact with the database and external services (email/SMS). Security is enforced via HTTPS and token-based authentication.

2.2 Product Functions

The main functions of the system are:

User Account Management:

Create and manage user accounts. Students, faculty, and company users register through a web form and receive role-based access. Admins (TPO/HoD) can add or remove users and assign roles. All users can update their profile information and password.

Enforce role-based permissions (RBAC). Each role has a customized dashboard and permitted actions. For example, students can only apply to jobs, whereas admins can approve job postings and schedule events.

Authentication and Security:

Secure login via email/password or institutional SSO.

Use Firebase Auth or an equivalent identity service with strong encryption.

Implement multi-factor authentication for admin roles if required.

Session management to prevent unauthorized access.

Company and Job Management:

Recruiters (Company users) self-register a company profile (name, website, contact).

Companies can post new job openings (position title, description, CTC, location, eligibility filters such as branches, CGPA cutoff, deadlines).

Admins approve or reject company registrations and job postings based on validation rules.

Companies can view lists of eligible applicants and enter interview results.

Student Profile and Application Processing:

Students create academic profiles (education history, CGPA, skills). Upload or build resume/CV within the portal.

Students browse available job listings. Only listings for which they meet eligibility criteria (checked automatically) are shown.

Students apply to jobs online; applications record timestamp and attached resume.

Faculty can endorse or recommend students for specific jobs (optional workflow).

Admins can filter and shortlist students by criteria (department, CGPA, skills) and notify them of opportunities.

Placement Drive Scheduling:

Admins schedule placement events (aptitude tests, interviews). Specify date, time slots, venues, participating companies.

Students select interview slot preferences based on availability (self-service scheduling).

System auto-confirms slots subject to capacity.

System updates student application status (e.g., "Scheduled", "Completed", "Result Pending").

Notifications and Communications:

Automatic email and/or SMS notifications at key events: registration confirmation, company approval, drive announcements, interview schedules, and placement results. (E.g. use SendGrid for email and Twilio for SMS as per feasibility recommendations.)

Bulk messaging tools for admins (e.g., send email to all eligible students of a branch).

In-app alerts on the dashboard for upcoming events or status changes.

Reports and Analytics:

Dashboards for admins/HoDs showing KPIs: number of registered companies, total students, applications per drive, success rates, historical trends.

Exportable reports (CSV/PDF) of student placements, company participation, etc.

Visualizations (charts) to help decision-making.

Content and Notice Management (Departmental):

Department-specific sections where faculty/admins can post internal notices, events, or seminar announcements visible only to that department’s students and faculty.

The above functions correspond to typical use cases (student applies for job, admin schedules drive, company reviews applicants, etc.). Each functional requirement will be detailed in section 3.3. In general, the system shall do *what* is needed (functional requirements) and also *perform well* under expected load (non-functional requirements)[1].

2.3 User Classes and Characteristics

The primary user classes are:

- **Students (Undergraduate/PG):** Typical age 18–25, generally tech-savvy and university-educated. Students use the portal to update profiles, browse/apply for jobs, and check placement statuses. They require a simple, intuitive UI (likely mobile-responsive). Expectations: ease of use, quick feedback on applications.
- **Faculty:** Professors and lecturers, comfortable with web portals. Faculty can log in to endorse students (optional) and view placement metrics (e.g. how many students from their department got placed). They may post departmental notices. Expectations: a departmental dashboard and reporting features.
- **Placement Administrators (TPOs, Dept Heads):** Responsible for the entire placement process. May have basic to advanced technical skill. They will use the full admin interface: managing users/companies, scheduling events, and viewing analytics. They require robust tools (e.g. filters, search, bulk email) and high reliability (downtime would disrupt operations).
- **Company Recruiters:** HR representatives or recruiters from companies. They register via a simple form and then can manage job postings and interview results. They need a straightforward interface for posting jobs and viewing applicants. Reliability and confidentiality (applicant info security) are important to them.
- **System Administrators:** (If separate from placement admins) IT staff who deploy and maintain the system infrastructure. They need technical documentation, monitoring tools, and database access. This role is mostly on the operation side and outside the main user interface.

Each user class has distinct needs and access levels, enforced via RBAC. The UI will present only the relevant features to each role (principle of least privilege). All users are assumed to have web access and basic computer literacy.

2.4 User Constraints

Accessibility: The portal must work on major web browsers (Chrome, Firefox, Edge, Safari) on desktop and mobile. It should use responsive design so that students on phones or tablets have a good experience. Adherence to WCAG 2.1 AA guidelines is recommended for accessibility (e.g., alt text, keyboard navigation).

Data Privacy: Must comply with applicable data protection policies (e.g. University or national privacy rules). Student personal data should not be exposed to unauthorized users. Only permitted attributes (CGPA, major) are used for eligibility checks.

Concurrency: The system should handle expected peak loads (e.g. when drive results are posted) without significant delay. (See Performance Requirements for specifics.)

Authentication: Users must use institutional email for login (to validate identity), unless using Firebase Social logins (e.g. Google SSO) as an option. Passwords are stored hashed; if Firebase Auth is used, security is managed by that service.

Resource Limitations: The university's servers or cloud budget may limit maximum server instances. The design assumes moderate traffic (thousands of users), not millions. Data growth estimates (e.g., 10,000 students, 500 companies, 100,000 applications per year) should fit within typical cloud storage quotas.

Third-party Services: The system relies on third-party services (e.g. Firebase Auth, email/SMS gateways). It assumes those services remain available (SLA compliant) and that any service limits (quota on emails, etc.) are sufficient or can be increased.

Browser Support: No support for obsolete browsers (like IE11). Ensure app works on evergreen browsers only.

Locale/Language: English is the primary language (en-GB spelling convention). No multilingual support is required initially, except for possible regional variations in date/time formats.

Compliance: If any regulatory constraints apply (e.g. PCI compliance if payment integration is ever needed, or FERPA if international student data is involved), they must be identified early (not currently in scope).

2.5 Assumptions and Dependencies

User Base: It is assumed that all students and faculty who use placements are registered in the University's student information system. Initial user accounts may

be imported or created by admin rather than self-registration (though sign-up flow is provided).

Network Infrastructure: Reliable internet and network infrastructure is assumed on campus and for users (students may have their own broadband). University will provide necessary network bandwidth and security (firewall rules) for hosting the portal.

Technologies: The development team is familiar with chosen technologies (React/Next.js, Node.js/Express, Django, Firebase). The feasibility study notes these are widely used technologies (React/Node/PostgreSQL) with abundant resources[7].

Cloud Services: It is assumed the university agrees to use cloud services like AWS/Azure/Google Cloud. If using Firebase, a Google Cloud project will be set up for the portal. As noted by Firebase docs, all apps under one project share resources (e.g. Auth, Firestore)[4], which simplifies integration.

Notifications: The university will provide or approve an email domain (e.g. @hyderabaduniversity.ac.in) and an SMS gateway (e.g. Twilio) for sending notifications.

Maintenance: The project assumes funding for at least 3 years of maintenance after deployment, including minor enhancements. Future major upgrades (e.g. mobile app development) are out of scope.

Security Standards: The team will follow industry security best practices (HTTPS, input validation, encryption), but the SRS assumes no unknown security vulnerabilities. A security audit may be performed during development (out of SRS scope).

3. Specific Requirements

3.1 Performance Requirements

Concurrency: The system **shall** support at least **2000 concurrent users** during peak periods (e.g. placement results announcements) without significant degradation of response time. It **should** be scalable to handle **5000** concurrent users with additional resources.

Response Time: Under normal load, 95% of web page requests **shall** have a server response time under **2 seconds**. Critical actions (login, form submission) **shall** complete in under **1 second** on average.

Throughput: The backend APIs **shall** process at least **100 requests per second** in aggregate at peak.

Uptime: The system **shall** achieve at least **99.5% uptime** (excluding scheduled maintenance) on a monthly basis.

Capacity: The database **shall** accommodate initial data for up to **10,000 students, 1,000 faculty, 2,000 companies, 100 active job postings, and 100,000 application records** without performance loss.

Scalability: The design **shall** allow horizontal scaling of web and API servers (e.g. multiple instances behind a load balancer). Database replication or sharding may be used if needed.

Data Freshness: Notifications (email/SMS) **shall** be sent within **5 minutes** of the triggering event (application submission, result entry, etc.).

Recovery: In case of a server failure, system restart **shall** occur within **1 hour**, and no more than **1%** of user data (in flight) **shall** be lost due to caching (with immediate consistency otherwise).

These performance numbers are illustrative; exact values may be refined. For example, a realistic target could be supporting **1,000 concurrent users** with sub-second response (as recommended in SRS examples[8]). The SRS provides room for growth. Load testing will verify these metrics during implementation.

3.2 Interface Requirements

3.2.1 User Interfaces

Web Interface: The system **shall** have a responsive web UI accessible at the university's placement portal URL. It **shall** adapt to desktop, tablet, and mobile screen sizes.

Dashboards: Each user role **shall** see a role-specific dashboard after login (students see job opportunities; admins see upcoming drives and stats; companies see applicant overviews). Dashboards **shall** display real-time data (e.g. number of applications) using charts or summaries.

Forms and Tables: Input forms (registration, job posting, application) **shall** validate fields client- and server-side. Tables (e.g. list of jobs, students) **shall** support pagination, sorting, and searching.

Notifications: Alerts **shall** appear in the UI for events (new message, upcoming interview). A persistent notifications dropdown **shall** be accessible.

Accessibility: The UI **shall** conform to WCAG 2.1 AA guidelines (color contrast, ARIA labels, keyboard navigation).

Browser Compatibility: The UI **shall** function correctly on current versions of Chrome, Firefox, Edge, and Safari.

3.2.2 Hardware Interfaces

Servers: The system will be deployed on standard cloud virtual machines (e.g. AWS EC2 or GCP Compute). Minimum configuration: quad-core CPU, 16GB RAM for the main application server. The database server will have comparable resources or use managed SQL/NoSQL service.

Client Devices: Users **shall** use devices (PC, laptops, tablets, smartphones) with at least 1GHz CPU and 1GB RAM, and a modern browser. The portal **shall** be usable on devices with screen width down to 320px (mobile portrait).

Network: The system **shall** function over standard internet connections (>= 1 Mbps). It **shall** degrade gracefully if connection slows (e.g. by queuing actions).

Printers/Scanners: Not applicable (system is digital only; no physical hardware interfaces required).

3.2.3 Software Interfaces

Authentication Service: The system **shall** use Firebase Authentication (or equivalent) for user login. It **shall** interact via OAuth2/JWT. The Firebase Admin SDK **shall** be used on the server to manage users programmatically.

Database: The backend **shall** interact with the database through an ORM or direct queries. If using Firebase (Firestore), the client and server SDKs **shall** connect to Firestore.

Email/SMS APIs: The system **shall** integrate with an email API (e.g. SendGrid) and an SMS API (e.g. Twilio) for notifications. These services are accessed via RESTful APIs using secure credentials.

Third-Party APIs: If single sign-on (SSO) is used, the system **shall** interface with the university's identity provider (e.g. LDAP/Active Directory or Google Workspace API).

External Data Sources: The system **shall** periodically import/update static data from university databases (e.g., department lists, course catalog) through CSV uploads or API, as part of administration tasks.

System Monitoring: The system **shall** provide logs (text files or a logging service) of errors and usage. These logs can be accessed by administrators or developers for troubleshooting.

3.2.4 Communication Interfaces

REST API: All server functions **shall** be exposed via a REST API over HTTPS. Endpoints will use JSON for requests and responses.

WebSockets (optional): For real-time updates (e.g. notifications), the system **may** use WebSockets or Firebase Realtime Database/Firestore listeners.

Security Protocols: All communications **shall** use HTTPS/TLS 1.2 or higher. Sensitive data (passwords, tokens) **shall** never be sent in cleartext.

Client-Server Interaction: The frontend **shall** authenticate via the API, receive a token, and attach it to subsequent calls (as an HTTP header) to identify the user's session.

3.3 Functional Requirements

Functional requirements define what the system must do. They are grouped by major feature:

1. **User Registration and Login:**
2. **FR1.1**: The system shall allow new users to register an account by providing name, email (university email preferred), department, and password.
3. **FR1.2**: The system shall send a verification email upon registration. Accounts remain inactive until verified.
4. **FR1.3**: The system shall allow existing users to log in with email and password. Alternatively, it shall support Google/Firebase SSO.
5. **FR1.4**: The system shall allow users to reset forgotten passwords via email link.
6. **Role-Based Access Control:**
7. **FR2.1**: The system shall assign a default role to each new user based on registration: Student, Faculty, or Recruiter. Admin accounts are created by existing admins.
8. **FR2.2**: The system shall enforce that users can only access pages and functions permitted by their role (e.g., only Admins can schedule drives).
9. **Student Profile Management:**
10. **FR3.1**: The system shall allow students to complete a profile: personal details, academic records (CGPA, degree), and department.

11. **FR3.2**: The system shall allow students to upload or build a resume/CV within the portal.
12. **FR3.3**: The system shall allow students to view a list of job openings for which they are eligible (based on criteria).
13. **FR3.4**: The system shall allow students to apply to an eligible job posting. Upon application, the student's status for that job is recorded (e.g. Applied, Shortlisted, Interview Scheduled).
14. **FR3.5**: The system shall allow students to view the status and history of all their job applications and interview schedules.

15. Faculty Functions:

16. **FR4.1**: The system shall allow faculty to view placement statistics for students of their department (e.g., number of placed students by batch).
17. **FR4.2**: The system shall allow faculty to endorse or recommend a student for a particular company or job opening (optional). Endorsements are visible to admins and the student's profile.
18. **FR4.3**: The system shall allow faculty to post academic or seminar notices on their department's page within the portal (visible to students/faculty of that department).

19. Administrator (TPO/HoD) Functions:

20. **FR5.1**: The system shall allow admins to add, edit, or delete user accounts (students, faculty, recruiters).
21. **FR5.2**: The system shall allow admins to create and manage placement drives (dates, participants, venue).
22. **FR5.3**: The system shall allow admins to approve or reject company registrations and job postings.
23. **FR5.4**: The system shall allow admins to shortlist or filter students (by CGPA, branch, skills) and send targeted notifications (e.g. email all eligible candidates).
24. **FR5.5**: The system shall allow admins to generate and view reports (charts, tables) on placement metrics (placement rate, number of drives, etc.).
25. **FR5.6**: The system shall allow admins to manage global site content (e.g. university-wide announcements, help pages).

26. Company/Recruiter Functions:

27. **FR6.1**: The system shall allow companies to register (name, website, HR contact). Registration is pending until an admin approves it.
28. **FR6.2**: The system shall allow approved companies to post one or more job openings with all details (title, description, CTC, location, eligibility filters, application deadline).
29. **FR6.3**: The system shall allow companies to view a list of student applications for each job posting, including student profiles and resumes of eligible candidates.
30. **FR6.4**: The system shall allow companies to update the status of each applicant (e.g. “Selected” or “Rejected”). Upon status change, the system will notify the student.

31. Scheduling and Interview Management:

32. **FR7.1**: The system shall allow admins to create time slots for interviews (date/time/venue/slots available) for each job or drive.
33. **FR7.2**: The system shall allow students to select or be assigned to interview slots for jobs they applied to.
34. **FR7.3**: The system shall automatically handle slot conflicts (e.g. no double-booking of a student or over-booking a slot).
35. **FR7.4**: The system shall allow the portal to automatically send reminders to students one day before their scheduled interview/test.
36. **FR7.5**: The system shall record and display the outcome of interviews (pass/fail) as entered by companies or admins.

37. Notifications and Messaging:

38. **FR8.1**: The system shall send email and/or SMS notifications for key events: registration success, job posting approvals, interview schedules, and results.
39. **FR8.2**: The system shall allow admins to send custom announcements (email) to selected user groups (e.g. all final-year students).
40. **FR8.3**: The system shall display in-app alerts or messages (with a notification icon) for new announcements or status changes.

41. Reporting and Analytics:

42. **FR9.1**: The system shall compile placement data and display dashboards of KPIs (e.g. total placed students, number of companies participated).

43. **FR9.2**: The system shall allow exporting reports (CSV, PDF) of placement outcomes and application records for further analysis.
44. **FR9.3**: The system shall refresh analytic data at least daily (cron job or on-demand refresh) to present up-to-date statistics.

45. **Data Management:**

    - **FR10.1**: The system shall maintain a database of all data (users, companies, jobs, applications) with proper foreign key relationships or reference links (in NoSQL).
    - **FR10.2**: The system shall enforce data validation rules (e.g. CGPA between 0 and 10, email format, mandatory fields).

These functional requirements form the core behaviors of the system. The examples above reflect standard SRS practice: “Users should be able to register and log in” or “Admins should be able to download reports”[9]. Ambiguous or implementation-specific language has been avoided. Each requirement is testable (e.g. one can verify that a student **can** log in after registration, or that notifications are sent).

3.4 Analysis Classes / Data Dictionary

Below is a high-level list of main classes (or database tables/collections) and their key attributes. These represent the core data entities of the system:

User (abstract): Attributes include *UserID (PK)*, *Name*, *Email*, *Role* (Student/Faculty/Admin/Company), *Department*, *PasswordHash*. All user roles inherit from this.

Student (extends User): Additional attributes: *CGPA*, *ResumeLink*, *Department*, *EnrollmentYear*.

Faculty (extends User): Additional: *Department*, *Designation*.

Administrator (extends User): Additional: *Department(s)Managed* (TPO or HoD).

Company (extends User): Additional: *CompanyName*, *Website*, *ContactPerson*, *Industry*, *CompanyDescription*.

JobPosting: *JobID (PK)*, *CompanyID (FK)*, *Title*, *Description*, *CTC*, *Location*, *EligibilityCriteria* (e.g. minimum CGPA, required branches), *PostingDate*, *ApplicationDeadline*.

Application: *ApplicationID (PK)*, *JobID (FK)*, *StudentID (FK)*, *DateApplied*, *Status* (Applied, Shortlisted, Selected, Rejected), *ResumeVersion*, *Remarks*.

InterviewSlot: *SlotID (PK)*, *JobID (FK)*, *DateTime*, *Venue*, *Capacity*, *BookedCount*.

Enrollment: (linking students to interview slots) *EnrollmentID*, *StudentID (FK)*, *SlotID (FK)*, *Status*.

Notification: *NotificationID (PK)*, *UserID (FK)*, *Message*, *CreatedTimestamp*, *ReadFlag*.

DepartmentNotice: *NoticeID (PK)*, *DeptID*, *Title*, *Content*, *PostedBy (UserID)*, *PostedDate*.

Department: *DeptID (PK)*, *Name*, *Description*. (Master data from university.)

Data Dictionary: Each attribute will have defined types (string, integer, date, etc.) and constraints (e.g. email format, CGPA range). All primary keys uniquely identify records. Foreign keys (FK) ensure referential integrity between tables (or analogous references in a NoSQL model). This dictionary will be elaborated in system design documents.

3.5 Logical Database Requirements

The system **shall** use a database to store all structured data. Options include:

Cloud Firestore (NoSQL): Collections corresponding to the classes above (e.g., `users`, `jobs`, `applications`, `notifications`). Firestore's flexible schema is suitable for evolving requirements. Google advises using Cloud Firestore for modern apps needing richer queries and scalability[5]. Security rules will enforce that only authorized users can read/write each collection. For example, a Firestore rule could restrict each student's document so that only that student (and admins) can modify it, while the resume subcollection is only writable by the student.

Relational Database (PostgreSQL): Alternatively, a PostgreSQL schema will have tables as listed in the data dictionary. Tables will use appropriate normalization. E.g. a join table for student endorsements by faculty. Indices will be created on frequently queried fields (e.g. JobID, StudentID).

Regardless of the choice, the following **logical requirements** apply:- Data integrity constraints (PK, FK, unique keys) shall enforce consistency (no orphaned references).- All user passwords **shall** be stored as secure hashes (e.g. bcrypt) if not using Firebase Auth.- The database **shall** keep logs or history where needed (e.g. audit tables for

critical actions like user deletions).- The data model **shall** allow easy retrieval of a student's history of applications and interview results (e.g. via joins or subcollections).

No direct connection from external entities (users) to data stores is allowed; all access goes through server processes. This keeps the system secure and allows an API layer to enforce business rules.

3.6 Design Constraints

Standards Compliance: The system **shall** follow relevant standards (IEEE for SRS; OWASP for security; WCAG for accessibility) to ensure quality.

Frameworks/Languages: The frontend **shall** be implemented in a modern JavaScript framework (React.js or Next.js). The backend **shall** use a proven platform (Node.js/Express or Django/Python). Use of Firebase services (Auth, Firestore, Cloud Functions) is expected for authentication and real-time features.

Security: All data transfers **shall** use HTTPS. Sensitive data **shall** be encrypted in transit and at rest (e.g. database encryption or managed service encryption). Authentication tokens **shall** expire (e.g. JWT tokens valid for <24 hours).

Coding Practices: Code **shall** be modular and documented. For example, REST API endpoints **shall** adhere to a consistent URL naming scheme (e.g. `/api/students`, `/api/jobs/{id}`).

Version Control: All source code **shall** be managed in a Git repository with a clear branching strategy. The SRS assumes Git or similar will track changes.

Documentation: The design **shall** include UML diagrams (class diagrams for data model, sequence diagrams for key processes). The SRS assumes these will be provided in design documents (Appendix references).

Privacy: Compliance with data privacy (e.g. GDPR if applicable) may constrain certain features (like data retention); these requirements **shall** be documented as business rules.

Third-Party Limits: If using Firebase or other cloud resources, the system **shall** be designed within any quotas or limits of the chosen plan (or plans should be up-scalable).

3.7 Software System Attributes

Following IEEE SRS guidelines, the system's quality attributes are explicitly stated. The main attributes are:

Reliability: The system **shall** operate continuously with minimal failures. Automated backups **shall** be taken nightly, and a disaster recovery plan **shall** allow restoration of services within one hour. Fault-tolerance (e.g. redundant servers) **shall** be implemented so that single-point failures do not crash the entire system.

Availability: The system **shall** be available 24/7 with **99.5% uptime**. Scheduled maintenance windows **shall** be communicated to users in advance. This ensures placement operations are not disrupted. High availability is crucial for enterprise portals, as noted in SRS best practices[7].

Security: The system **shall** enforce role-based access control (RBAC) to segregate duties. All user passwords **shall** be hashed (or managed by Firebase Auth). The system **shall** protect against common web vulnerabilities (SQL/NoSQL injection, XSS, CSRF). Sensitive student or company data **shall** be encrypted at rest (database encryption) and in transit (HTTPS/TLS). Audit logs **shall** record administrative actions. According to SRS best practice, security requirements must be detailed to avoid guesswork[7].

Maintainability: The system **shall** be modular (separating UI, API, data layers). Code should follow coding standards and be well-documented. Automated tests **shall** be developed for critical components. This ensures ease of fixing bugs and adding features. Using managed services like Firebase can improve maintainability since Google handles infrastructure updates.

Portability: The system **shall** be deployable on any major cloud platform. For example, Docker containers **could** be used so the application can run on AWS, Azure, or GCP. The only platform-specific dependency is the web browser on the client side (no native OS dependencies). The use of web technologies (React, Node) makes the solution cross-platform.

Usability: (Additional attribute) The system **shall** have an intuitive interface. The SRS notes usability since it impacts user acceptance. For instance, first-time users **shall** be able to register or log in without training. Error messages **shall** be clear and instructive. These usability considerations, while not always listed in IEEE headers, are part of delivering a high-quality SRS.

Performance Efficiency: (Derived from performance requirements) The system **shall** meet the specified performance metrics to ensure a smooth user experience. Proper indexing, caching, and optimized queries **shall** be used to meet response time goals.

All the above attributes correspond to industry-standard quality factors (often listed as reliability, availability, security, maintainability, portability)[10]. The system's architecture and technologies (e.g. cloud hosting, Firebase) are chosen to support these attributes. For example, using Firebase/Cloud services inherently provides high availability and scalability, contributing to reliability and performance.

4. Support Information

Development Environment: The system development **shall** use version control (Git), and deployment **shall** use CI/CD pipelines. (The SRS assumes developers will set this up.)

Testing: An SRS review **shall** be conducted with stakeholders. Test plans will be derived from this SRS to verify each requirement. This includes unit, integration, and system testing, as well as security testing.

Documentation: User manuals and system documentation **shall** be provided. The SRS references UML diagrams and process flows which **shall** be included in the design documentation.

Training: Brief user guides or tutorials **shall** be created for first-time users (outside the SRS scope).

Appendix A (Glossary): A glossary of terms and acronyms (as in section 1.3) will be maintained.

Appendix B (Use Case Diagram): A high-level use case diagram shows interactions (e.g., Student applies for job, Company reviews applications).

Appendix C (Data Dictionary): Detailed data model fields for each entity.

Future Considerations: Future work might include a mobile app interface, advanced analytics, or integration with other campus systems. These are noted here but not required in this SRS.

This SRS is intended for review by all stakeholders (students, faculty, TPOs, recruiters, developers). It provides a clear baseline: every requirement is testable and based on project needs. Ambiguities or omissions should be resolved before design begins to avoid costly rework.

References

Karl E. Wiegers, *"Software requirements specification and IEEE standards"*, TechTarget, 2007[3][1].

Wikipedia, *"Software requirements specification"* (entry on requirement quality) [2].

Google Firebase Documentation, *"General best practices for setting up Firebase projects"*[4].

Google Firebase Documentation, *"Cloud Firestore"* (Firebase NoSQL database) [5].

BMC Software Blog, *"Software Requirements Specification (SRS) – How to Write SRS with Examples"*[6].

---

[1] [3] [7] Software requirements specification and IEEE standards | TechTarget

https://www.techtarget.com/searchsoftwarequality/answer/Software-requirements-specification-and-the-IEEE-standard

[2] Software requirements specification - Wikipedia

https://en.wikipedia.org/wiki/Software_requirements_specification

[4] General best practices for setting up Firebase projects | Firebase Documentation

https://firebase.google.com/docs/projects/dev-workflows/general-best-practices

[5] Firebase Realtime Database

https://firebase.google.com/docs/database

[6] [10] SRS: Software Requirement Specifications Basics – BMC Software | Blogs

https://www.bmc.com/blogs/software-requirements-specification-how-to-write-srs-with-examples/

[8] [9] Guide to Writing Software Requirements Specification (SRS)

https://www.modernrequirements.com/blogs/requirements-specification/